

System Vitals — Complete Technical Reference

Vermont Health Platform · HTR
Feature: Bed Capacity, Transfer Routing & Repatriation Dashboard
Route: /system-vitals
Status: Live · Synthetic Data (demo mode)
Last Updated: April 29, 2026

Table of Contents

- 1. [What is System Vitals?](#)
- 2. [End-User Guide](#)
- 3. [Architecture Overview](#)
- 4. [Data Layer](#)
- 5. [Component Reference](#)
- 6. [Algorithm System](#)
- 7. [Drive-Time Matrix](#)
- 8. [Navigation & Routing Integration](#)
- 9. [Data Consistency Architecture](#)
- 10. [How to Update Data](#)
- 11. [Relationship to the Act 167 Simulator](#)
- 12. [Roadmap: From Synthetic to Live Data](#)
- 13. [File Map](#)

1. What is System Vitals?

System Vitals is an operational situational-awareness tool for Vermont's 14-hospital network. It gives transfer coordinators, hospital administrators, and policy analysts a real-time (or near-real-time) view of three critical questions:

- 1. **Where is there capacity?** — Which hospitals have open beds, by unit type?
- 2. **Where should I send a patient?** — Given a sending hospital, acuity level, and specialty need, which receiving hospital is the best match right now — and by which ranking logic?
- 3. **Who can go home?** — Which patients at tertiary centers are past their target length of stay and have a bed confirmed at their home hospital — and in what order should they be prioritized?

The feature covers Vermont's full continuum: two tertiary centers (UVMHC, Dartmouth-Hitchcock), three regional hospitals (CVMC, SVMC, RPMC), and nine critical access hospitals.

2. End-User Guide

Getting There

Navigate to **States & Programs** → **System Vitals** in the left sidebar, or via the **STATES & PROGRAMS** menu in the top navigation bar. Breadcrumb: **Home / System Vitals**.

The Ticker Bar (top of the System Vitals panel)

A row of colored chips — one per hospital (first 8 shown):

● UVMHC: 17 avail ● DHMC: 30 avail ● CVMC: 32 avail ...

Color is based on **total available beds across all unit types**:

Color	Threshold	Meaning
Green	> 15 avail	Healthy
Amber	6–15 avail	Moderate pressure
Red	≤ 5 avail	Critically constrained

These numbers are computed from the same data source as the capacity grid — they always match.

Tab 1: Capacity Grid

Shows a card for every hospital. No algorithm selector on this tab — it is a pure display of current census data.

Each card shows:

- Hospital name, type (Tertiary / Regional / Critical), county
- One bar per unit type: `available` / `total` with color-coded fill
- An overall status badge

Unit types:

Code	Full name
ICU	Intensive Care Unit
Med-surg	Medical-Surgical
Behav.	Behavioral Health
SNF/step-down	Skilled Nursing / Step-down

If a hospital has 0 total beds of a type, that row is omitted from the card.

Bar color thresholds:

Color	Available %	Meaning
Green #639922	> 20%	Available
Amber #BA7517	5–20%	Limited
Red #E24B4A	< 5%	Critical
Gray #B4B2A9	0 total beds	Unit not present

Overall status badge reflects the *worst* unit (most conservative signal).

Filters & sort:

- Filter: All / Critical access only / Regional only / Tertiary only
- Sort: Name (A–Z) / Most available / Highest stress

Clicking a card expands a detail pane showing total beds, available now, occupancy %, pending discharge estimate (synthetic: 30% of available), specialties on staff, and transfer center status.

Surge banner appears when UVMMC ICU has ≤ 2 beds available.

Tab 2: Transfer Routing

Use this tab to find the best receiving hospital for a specific patient transfer.

The Algorithm Selector

At the top of the Transfer Routing tab is the **Ranking Algorithm** selector — a dropdown with a description line that explains exactly what each algorithm prioritizes:

Algorithm	What it prioritizes	Best for
Best overall match	Balanced: bed availability + specialty + hospital tier + transfer center	General use, uncertain situation
Specialty first	Specialty match weighted ~2x above bed availability	Complex/subspecialty cases
Closest available bed	Drive-time proximity to the sending hospital	Time-sensitive, stable transfers
Most available capacity	Highest open-bed percentage in the requested unit	System-wide surge, need a reliable bed fast
Conserve tertiary capacity	Prefers regional/critical over tertiary; penalizes tertiary placement	Step-down patients who don't need tertiary resources

The selected algorithm name appears as an italic label ("Ranked by: Specialty first") next to the results header, so users always know which logic produced the list they are reading.

Transfer Inputs

Field	Options
Sending hospital	Any of the 14 hospitals (excluded from results)
Acuity level	ICU / Med-surg / Behavioral / SNF
Specialty needed	Any / Cardiac / Neurology-stroke / Orthopedics / Psychiatry / Pediatrics / Oncology
Transport available	Ground / Air / Either (displayed; not yet factored into scoring)

Press **Find best match** ↗ to run the selected algorithm.

Results

Up to 4 hospitals returned, ranked #1–#4. Each result shows:

- Rank badge (green=1st, blue=2nd, gray=3rd/4th)
- Hospital name and county
- **Match tags** — blue = positive factor, gray = neutral/negative:
 - N [acuity] beds — beds available of requested type
 - Specialty name or no [specialty]
 - transfer center or direct admit
 - Hospital type (tertiary / regional / critical)
 - ~N min drive — estimated drive time from sending hospital (blue if < 60 min)
- **Match score** (0–100, scale varies by algorithm)

Tab 3: Repatriation Queue

Shows patients currently at a tertiary center who are past their clinical target length of stay and ready to return to their home hospital.

The Algorithm Selector

The Repatriation Queue has its own dedicated algorithm selector — separate from the Transfer Routing selector because the ranking context is different:

Algorithm	What it prioritizes	Best for
Most overdue first	Days past clinical target LOS	Clearing the most urgent LOS backlog
Bed-confirmed first	Patients whose home hospital already has a bed available	Completing transfers that are logistically ready now
Shortest transfer distance	Patients whose home hospital is geographically closest	Minimizing transport burden
Combined priority score	Weighs days overdue + bed availability + proximity together	Balanced queue management

Each patient row shows:

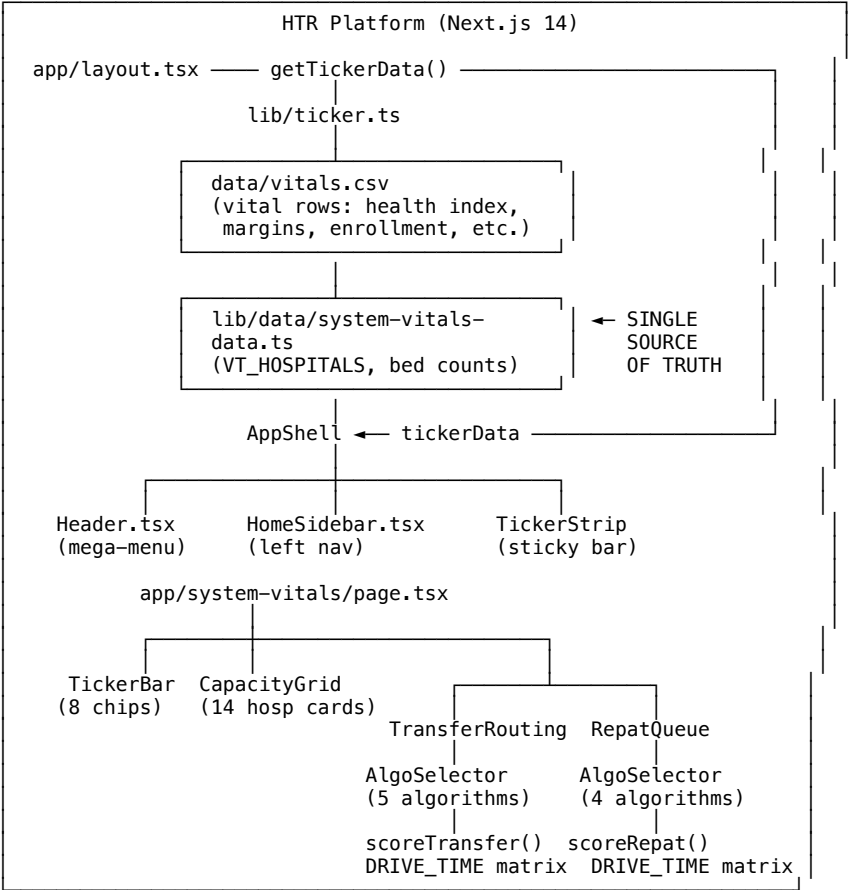
- Name, age, diagnosis
- Current location → home hospital, acuity, specialty
- LOS vs. target, days over target
- Estimated drive time to home hospital
- Current blocker (reason for delay)
- Home hospital bed availability in the required unit (green = available, red = none)

"Initiate transfer" button is active (blue) only when a bed is available. Clicking opens a two-step confirm dialog.

The Global Sticky Ticker Strip

The thin strip at the top of every HTR page shows bed availability chips for Vermont hospitals. These numbers come from the **same data source** as the System Vitals capacity grid — they are always identical.

3. Architecture Overview



Key principle: One copy of hospital bed data. Everything reads from lib/data/system-vitals-data.ts. Algorithms run entirely client-side — no API calls.

4. Data Layer

Primary File: frontend/lib/data/system-vitals-data.ts

This is the **only** file to edit when changing bed availability numbers.

Types

```
type BedKey = "icu" | "medsurg" | "behavioral" | "snf";

interface BedCounts {
  total: number; // licensed/tracked beds in this unit
  avail: number; // currently available (unoccupied and ready)
}

interface VTHospital {
  id: string;           // url-safe slug, e.g. "uvmmc"
  name: string;         // full display name
  short: string;        // ticker abbreviation, e.g. "UVMC"
  type: "tertiary" | "regional" | "critical";
  region: string;       // Vermont county or sub-region
  beds: Record<BedKey, BedCounts>;
  specialties: string[]; // ["cardiac","neuro","ortho","psych","peds","oncology","any"]
  transfer_center: boolean;
}

interface RepatPatient {
  id: string;
  name: string;      // anonymized, e.g. "Patient A"
  age: number;
  dx: string;        // diagnosis
  at: string;        // current facility short name: "UVMC" or "DHMC"
  home: string;      // home hospital short name (must match VTHospital.short or substring of name)
  los: number;       // actual length of stay (days)
  target_los: number; // clinical target LOS
  days_over: number; // los - target_los
  acuity: BedKey;    // bed type needed at home hospital
  specialty: string;
  blocker: string;   // current coordination blocker
}
```

Exports

Export	Type	Description
VT_HOSPITALS	VTHospital[]	All 14 Vermont hospitals with bed data
REPAT_PATIENTS	RepatPatient[]	6 synthetic repatriation-ready patients
totalAvail(h)	(VTHospital) → number	Sum of avail across all bed types
totalBeds(h)	(VTHospital) → number	Sum of total across all bed types

Secondary File: frontend/data/vitals.csv

Two row types:

- **type: vital** — manually maintained stats for the global ticker (health index, margins, workforce gap, etc.). Edit freely.
- **type: bed** — **ignored at runtime**. The ticker engine strips these out and replaces them with rows computed from VT_HOSPITALS. Do not edit bed rows in the CSV.

Tertiary Reference: frontend/app/vermont-act-167/simulator/data.ts

A separate Hospital interface with beds: number (total licensed beds, e.g. 562 for UVMC) used by the Act 167 financial/policy simulator. **Do not merge with System Vitals data** — they serve different purposes.

5. Component Reference

SystemVitalsPage (default export)

State:

State variable	Type	Default	Purpose
activeTab	"capacity" "transfer" "repatriation"	"capacity"	Active tab
transferAlgo	TransferAlgoId	"balanced"	Shared transfer ranking algorithm
repatriationAlgo	RepatriationAlgoId	"days_over"	Shared repatriation sort algorithm

The algorithm state lives at the page level and is passed down as props. This means algorithm selection persists when switching tabs and returning.

AlgoSelector<T>

Generic algorithm selector widget, used by both Transfer Routing and Repatriation Queue.

Props:

```
{
  algos: AlgoDef<T>[]; // list of algorithm definitions
  value: T;             // currently selected algorithm id
  onChange: (v: T) => void; // callback when user changes selection
}
```

Renders:

- "RANKING ALGORITHM" label
- <select> with one <option> per algorithm
- Description paragraph below the select, updating live as selection changes

TransferRouting

Props: { algo: TransferAlgoId; onAlgoChange: (v: TransferAlgoId) => void }

State: fromId, acuity, specialty, results

Calls scoreTransfer() on button click. Results stored in component state — no API call.

RepatriationQueue

Props: { algo: RepatAlgoId; onAlgoChange: (v: RepatAlgoId) => void }

State: filter (facility), confirmId

Calls scoreRepat() on every render (no button — re-sorts live as algorithm changes). For each patient, looks up the home hospital in HOSPITALS to get current bed availability in the required acuity type.

CapacityGrid

No algorithm selector. State: filter, sort, selected. Purely displays VT_HOSPITALS data.

HospCard / DetailPane / TickerBar

Pure display components — no algorithm involvement.

6. Algorithm System

Algorithm Type Definitions

```
type TransferAlgoId =
| "balanced"           // Best overall match
| "specialty_first"    // Specialty first
| "closest"            // Closest available bed
| "capacity_first"     // Most available capacity
| "conserve_tertiary"; // Conserve tertiary capacity

type RepatAlgoId =
| "days_over"         // Most overdue first
| "home_bed_ready"     // Bed-confirmed first
| "closest_home"       // Shortest transfer distance
| "combined";          // Combined priority score
```

Transfer Scoring — scoreTransfer()

scoreTransfer(candidates, fromId, acuity, specialty, algo) → ScoredHospital[]

Step 1 — Filter candidates:

- Remove the sending hospital (h.id === fromId)
- Remove any hospital with 0 available beds of the requested acuity type

Step 2 — Compute per-hospital variables:

```
bedAvail = h.beds[acuity].avail
bedTotal = h.beds[acuity].total
bedPct   = bedAvail / bedTotal
hasSpec  = (specialty === "any") OR (h.specialties includes specialty) OR (h.specialties includes "any")
driveMin = getDriveTime(fromId, h.id) // from DRIVE_TIME matrix
```

Step 3 — Score by algorithm:

Algorithm	Formula
balanced	bedPct×40 + hasSpec?35:0 + tier(15/10/0) + tc?10:0
specialty_first	hasSpec?60:0 + bedPct×25 + tier(10/5/0) + tc?5:0
closest	max(0, 100-(driveMin/180)×100)×0.7 + hasbed?20:0 + hasSpec?10:0
capacity_first	bedPct×70 + hasSpec?20:0 + tc?10:0
conserve_tertiary	type==critical?30 : type==regional?20 : 0 + bedPct×40 + hasSpec?20:0 + tc?10:0

Where tier = 15 for tertiary, 10 for regional, 0 for critical. tc = transfer_center.

Step 4: Sort descending, return top 4.

Returned type:

```
type ScoredHospital = Hospital & {
  score: number;
  hasSpec: boolean;
  bedAvail: number;
  bedTotal: number;
  driveMin: number;    // always included, shown as match tag
};
```

Repatration Scoring — scoreRepat()

scoreRepat(patients, algo) → RepatPatient[]

Per-patient variables computed internally:

```
homeH      = HOSPITALS.find(h => h.short === p.home || h.name.includes(p.home))
bedAvail    = homeH ? homeH.beds[p.acuity].avail : 0
tertId      = p.at === "UVMHC" ? "uvmmc" : "dhmc"
homeId      = homeH?.id ?? ""
driveMin    = getDriveTime(tertId, homeId)
```

Scoring by algorithm:

Algorithm	Formula
days_over	p.days_over × 10
home_bed_ready	bedAvail > 0 ? 100 + p.days_over : 0
closest_home	max(0, 200 - driveMin)
combined	(p.days_over × 20) + (bedAvail > 0 ? 40 : 0) + max(0, 40 - driveMin/5)

Returns patients sorted descending by score. The _score property is stripped before return — only RepatPatient fields are exposed.

Score Interpretation

Transfer match score — scale varies by algorithm:

- balanced, specialty_first: max ~100. Score of 75+ = strong match.
- closest: max ~100 (dominated by proximity). Score of 70+ = within ~45 min.
- capacity_first: max 100. Score of 60+ = unit is >85% open.
- conserve_tertiary: max ~100. High score = regional/critical with available bed.

Repatration score is used only for sort order. The absolute number is not displayed to users — only the resulting queue order matters.

7. Drive-Time Matrix

DRIVE_TIME is a hardcoded record in page.tsx based on approximate Vermont road travel times in minutes. It is symmetric — only one direction is stored per pair; getDriveTime(a, b) checks both DRIVE_TIME[a][b] and DRIVE_TIME[b][a], returning 120 as a conservative default if neither is found.

Key distances (minutes):

From	To	Min
UVMHC	CVMC	40
UVMHC	RRMC	65
UVMHC	Copley	55
UVMHC	Porter MC	35
UVMHC	DHMC	105
UVMHC	NVRH	95
DHMC	Springfield	50
DHMC	Mt. Ascutney	40
RRMC	Springfield	60
Brattleboro	Grace Cottage	20

To update: Edit the DRIVE_TIME constant at the top of app/system-vitals/page.tsx. The matrix is used by both the closest transfer algorithm and the closest_home / combined repatriation algorithms.

Production path: Replace with a call to VTRANS or Google Maps Distance Matrix API, keyed by hospital lat/lng from the Act 167 simulator's data.ts.

8. Navigation & Routing Integration

Left Sidebar (HomeSidebar.tsx)

Under States & Programs, between "Vermont RHT Program" and "AHEAD Model":

```
{ href: "/system-vitals", label: "System Vitals", icon: TableCellsIcon }
```

/system-vitals is in statesPrefixes — navigating to the page auto-expands the States & Programs accordion.

Header (Header.tsx)

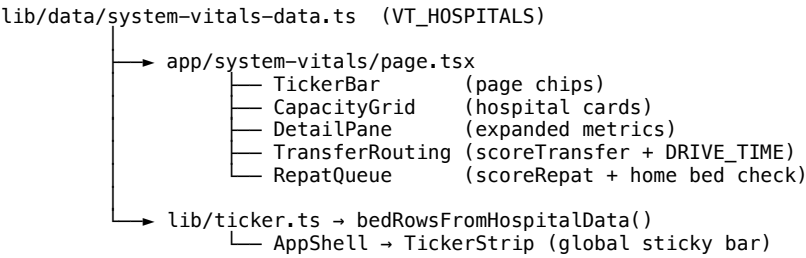
- **Desktop mega-menu:** In StatesPanel, between "Vermont Act 167" and "California CalAIM"
- **Mobile menu:** In the STATES & PROGRAMS accordion
- **Active highlight:** /system-vitals in the activeCheck string for the STATES & PROGRAMS button

9. Data Consistency Architecture

The Problem

The global sticky ticker strip and the System Vitals capacity grid both display bed availability. When they read from different sources they show different numbers — a trust-destroying inconsistency for clinical users.

The Solution



getTickerData() in lib/ticker.ts:

1. Reads data/vitals.csv, keeps only type: vital rows
2. Calls bedRowsFromHospitalData() which imports VT_HOSPITALS and computes bed rows dynamically
3. Returns both concatenated

Result: Architecturally impossible for the global ticker and the capacity grid to show different numbers.

10. How to Update Data

Bed availability numbers

Edit frontend/lib/data/system-vitals-data.ts — the VT_HOSPITALS array. Change the avail field for the relevant hospital and bed type. Save — the change propagates to the capacity grid, page ticker, transfer scoring, repatriation queue bed checks, and the global sticky strip simultaneously.

Vital statistics (non-bed)

Edit frontend/data/vitals.csv — type: vital rows only. Restart dev server to pick up changes.

Drive-time estimates

Edit the DRIVE_TIME constant at the top of app/system-vitals/page.tsx.

Add a hospital

Add to VT_HOSPITALS in system-vitals-data.ts. Add its row(s) to DRIVE_TIME in page.tsx.

Add a repatriation patient

Add to REPAT_PATIENTS in system-vitals-data.ts. The at field must be "UVMHC" or "DHMC". The home field must match VTHospital.short or a substring of VTHospital.name.

Add a new transfer algorithm

1. Add the id to TransferAlgoId union type in page.tsx
2. Add a AlgoDef entry to TRANSFER_ALGOS array (id, label, description)
3. Add a scoring branch in scoreTransfer() for the new id
4. Done — the AlgoSelector renders it automatically

Add a new repatriation algorithm

Same pattern: add to RepatAlgoId, REPAT_ALGOS, and scoreRepat().

11. Relationship to the Act 167 Simulator

Feature	Purpose	Data used
Act 167 Simulator	Policy & financial scenario modeling	simulator/data.ts — beds: 562 (licensed), financial metrics, Oliver Wyman recommendations

Feature	Purpose	Data used
System Vitals	Operational situational awareness + transfer decision support	system-vitals-data.ts — {icu, medsurg, behavioral, snf} with {total, avail}

Keep these datasets separate. They will be wired to different live data sources when production integration occurs.

12. Roadmap: From Synthetic to Live Data

Phase 1 — Current (Demonstration)

Static VT_HOSPITALS array. All algorithms fully functional. Drive-time matrix hardcoded.

Phase 2 — Database-backed (3–6 months)

Replace static array with PostgreSQL query. Schema: hospital_beds(hospital_id, bed_type, total, avail, updated_at). page.tsx becomes a server component that fetches and passes props to existing client components. No UI changes required.

Phase 3 — VITL HL7 ADT feed (6–18 months)

Connect to Vermont Information Technology Leaders HIE. HL7 v2 ADT messages (A01/A02/A03) update the bed table in real time with ~15–30 min latency.

Phase 4 — FHIR R4 real-time (18+ months)

Subscribe to FHIR Location resources via Server-Sent Events. Sub-5-minute latency. Enable transport field in transfer scoring via VTRANS routing API.

13. File Map

Vermont-Health-Platform/	
├── frontend/	
│ ├── app/	
│ │ └── system-vitals/	
│ │ └── page.tsx	← All UI + algorithm logic + DRIVE_TIME matrix
│ ├── components/	
│ │ ├── Header.tsx	← System Vitals in mega-menu + mobile menu
│ │ ├── HomeSidebar.tsx	← System Vitals in States & Programs
│ │ ├── AppShell.tsx	← Passes tickerData to TickerStrip
│ │ └── TickerStrip.tsx	← Renders bed chips in global sticky bar
│ ├── lib/	
│ │ ├── data/	
│ │ │ ├── system-vitals-data.ts	← SINGLE SOURCE OF TRUTH for bed data
│ │ │ └── ticker.ts	← Computes global ticker from VT_HOSPITALS
│ │ └── data/	
│ │ └── vitals.csv	← Non-bed vital statistics only
├── SYSTEM_VITALS_DOCUMENTATION.md	← This file
├── SYSTEM_VITALS_WHITEPAPER.md	← Stakeholder white paper
└── htr_bed_capacity_transfer_dashboard.html	← Original HTML reference

All bed availability and patient data are synthetic. No actual patient information is used or displayed.
Vermont Health Platform · April 29, 2026